UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

LENGUAJES Y COMPILADORES (LAPSO 2026-II)

Defensa Audiovisual - Prueba Diagnóstica (10%)

Equipo: ParseCore

"Estructurando la sintaxis, optimizando la ejecución."

Estudiante: Brayan Carreño | C.I.: V-32.015.527 | Sección: 01

Docente: Msc. Félix Márquez | Repositorio: github.com/Bryd3n/lyc-parsecore

Lineamientos y Estructura de la Evaluación

Ponderación Total: 10% (5% Código Fuente y Métricas + 5% Video de Defensa Audiovisual)

1. Programas y Código (5%)

- Codificación en C y Python con medición de tiempos para n=100.
- Resultados exportados a archivos comparativa_rendimiento.txt.
- Scripts de compilación y ejecución para entorno Windows.
- Repositorio estructurado en PRUEBA DIAGNOSTICA/Brayan_Carreno/.

2. Video de Defensa (5%)

- Duración máxima estricta de 5 minutos (formato horizontal 16:9).
- Presencia visible y continua del estudiante ante cámara web.
- Introducción obligatoria con Nombre, Cédula, Sección, Grupo y Eslogan.
- Demostración práctica de compilación, ejecución y benchmarks.

3. Política de Integridad e IA

- Dominio pleno y justificación matemática de cada algoritmo.
- El estudiante es un sujeto crítico, activo y analítico.
- Pruebas de verificación contra los 9 casos de prueba oficiales.
- Almacenamiento público en Google Drive vía video_defensa.txt.

Ejercicio 1: Validación de Movimientos de Caballo

Tablero de Ajedrez Estándar 8x8 [a-h][1-8] con Notación Algebraica (Prefijo K)

Fundamento Matemático del Salto en L

Desplazamiento geométrico entre (x1, y1) y (x2, y2):

$\Delta x = |x2 - x1|$, $\Delta y = |y2 - y1|$

Condición geométrica invariante:

$\Delta x^2 + \Delta y^2 = 5 \iff (\Delta x \cdot \Delta y = 2 \text{ y } \Delta x + \Delta y = 3)$

Detección Temprana de Errores (Cortocircuito):

1. Letra no permitida: Pieza != 'K' (ej: Qb4)
   -> 'movimiento INCORRECTO falla letra Q no pertenece a un caballo.'
2. Movimiento fuera de límites o geométricamente ilegal:
   -> 'INCORRECTO falla en el movimiento [i].'
3. Secuencia íntegra y válida:
   -> 'movimiento CORRECTO.'

Verificación contra Casos Oficiales del PDF

[OK] Caso 1 (Ak=a1, n=4): Kc2 Kd4 Kb3 Kc5
-> movimiento CORRECTO.

[OK] Caso 2 (Ak=a1, n=4): Kc2 Kd4 Kb4 Kc5
-> INCORRECTO falla en el movimiento 3.

[OK] Caso 4 (Ak=a2, n=5): Kc8 Ke4 Kf6 Ke8 Kg7
-> INCORRECTO falla en el movimiento 1.

[OK] Caso 8 (Ak=a1, n=10): ... Kc6 Qb4
-> movimiento INCORRECTO falla letra Q...

[OK] Caso 9 (Ak=a1, n=10): ... Kd8 Kh6 Kb4
-> INCORRECTO falla en el movimiento 9.

Ejercicio 1: Métricas de Rendimiento Multilenguaje

Comparativa de Tiempos de Ejecución para n = 100 y 10x 500 Movimientos (C vs Python)

Métrica / Escenario	Lenguaje C (GCC -O3)	Python 3.12 (CPython)	Factor de Aceleración
Tiempo n = 100 movimientos	**2.397 microsegundos**	36.730 microsegundos	**15.3x más veloz en C**
Prueba 10x 500 movimientos	**0.1254 ms (total)**	1.7528 ms (total)	**14.0x más veloz en C**
Promedio por 500 movimientos	**0.0125 ms / iteración**	0.1753 ms / iteración	**14.0x más veloz en C**

Análisis de Compilación y Arquitectura de Ejecución:

1. Acceso Directo a Memoria en C: La tokenización con strtok y la manipulación de punteros nativos operan directamente en memoria contigua sin capas de abstracción.
2. Optimización a Nivel de Registros: El compilador GCC con bandera -O3 mapea las variables de control (dx, dy, col, row) directamente a registros de CPU (EAX, EDX).
3. Sobrecarga del Intérprete CPython: Python requiere evaluación de bytecode en el bucle de evaluación de evaluación de CPython y la instanciación de objetos inmutables str en cada salto.

Ejercicio 2: Polinomio (x+1)^n y Pascal Dinámico

Generación de Coeficientes Binomiales con Asignación Dinámica Estricta en Memoria

Gestión de Memoria Dinámica (Heap)

Requerimiento central del profesor:

'Implemente el algoritmo utilizando memoria dinámica.'

Ciclo de Vida de los Punteros en C:

1. Reserva inicial: malloc(1 * sizeof(BigInt)) para la fila n=0.

2. Iteración de niveles: Para el nivel i, se reserva malloc((i+1) * sizeof(BigInt)).

3. Construcción combinatoria:
nueva_fila[j] = fila_actual[j-1] + fila_actual[j]

4. Prevención de Memory Leaks: Se invoca free(fila_actual) antes de reasignar el puntero fila_actual = nueva_fila.

5. Cero fugas: La memoria no utilizada se devuelve al sistema operativo de inmediato.

Evaluación Paso a Paso: f(x) = (x+1)^n

Ejemplo con n = 4 y x = 2.0:
Polinomio expandido: (x+1)^4 = x^4 + 4x^3 + 6x^2 + 4x + 1

- k=4: 1 · (2.0)^4 = 1 · 16.0 = 16.0
- k=3: 4 · (2.0)^3 = 4 · 8.0 = 32.0
- k=2: 6 · (2.0)^2 = 6 · 4.0 = 24.0
- k=1: 4 · (2.0)^1 = 4 · 2.0 = 8.0
- k=0: 1 · (2.0)^0 = 1 · 1.0 = 1.0

Suma por expansión polinomial: 81.0
Verificación directa (2 + 1)^4 = 3^4 = 81.0
Error de cálculo: 0.00e+00 (Exactitud Absoluta)

Ejercicio 2: El Reto de n = 100 y Desbordamiento

Manejo de Números Gigantes de 29 Dígitos en C (BigInt) vs Bignum Nativo en Python

El Límite Físico de 64 Bits:

- En C, el tipo unsigned long long almacena hasta 2^64 - 1 ≈ 1.84 × 10^19 (19 dígitos), desbordando en n >= 68.
- Para n = 100, el coeficiente central C(100, 50) posee 29 dígitos decimales:
  C(100, 50) = 100.891.344.545.564.193.334.812.497.256 (~1.0089 × 10^29)
- Solución implementada: Arquitectura BigInt dinámica en C con suma con acarreo y almacenamiento decimal exacto.

Parámetro	Lenguaje C (malloc/free)	Python 3.12	Observación Técnica
Tiempo generación fila n=100	184.431 microsegundos	194.370 microsegundos	C gestiona memoria y BigInt
Precisión en C(100, 50)	29 dígitos exactos	29 dígitos exactos	Consistencia numérica total

Ejercicio 3: Analizador Léxico (Traductor C a Español)

Carga Dinámica en RAM y Autómata Finito Determinista para Reconocimiento de Tokens

1. Carga Dinámica en Heap

- Apertura de archivo en modo binario.
- Medición exacta de bytes con fseek() y ftell().
- Reserva dinámica con malloc(tamaño + 1).
- Lectura en bloque a memoria con fread().
- Liberación final garantizada con free().

2. Autómata Léxico (DFA)

- Escaneo caracter a caracter sin buffer estático.
- Estado 1: Literales de cadena "..." (Escape \).
- Estado 2: Literales de carácter '...'.
- Estado 3: Comentarios de línea // y bloque /* */.
- Estado 4: Identificadores [a-zA-Z_][a-zA-Z0-9_]*.

3. Reglas de Compilación

- Preservación Estricta: Palabras clave dentro de strings o comentarios NO se tocan.
- Integridad de Identificadores: Variables como 'contador_while' no se rompen.
- Tabla de equivalencias para 25 palabras reservadas (if->si, int->entero...).
- Reporte estadístico de tokens.

Ejercicio 3: Demostración Práctica del Traductor

Contraste Visual: Código Original en C vs Código Traducido a Español y Estadísticas

Código C Original:

```
int calcular(int n) {
    if (n <= 1) {
        return 1;
    } else {
        return n * calcular(n - 1);
    }
}

void loop() {
    while (contador < 5) {
        contador++;
    }
    for (int i=0; i<3; i++) {
        char* m = "Cadena if else";
    }
}
```

Código Traducido a Español:

```
entero calcular(entero n) {
    si (n <= 1) {
        retornar 1;
    } sino {
        retornar n * calcular(n - 1);
    }
}

vacio loop() {
    mientras (contador < 5) {
        contador++;
    }
    para (entero i=0; i<3; i++) {
        caracter* m = "Cadena if else";
    }
}
```

Métricas Léxicas:

- Bytes procesados:
  837 bytes en RAM
- Identificadores:
  43 leídos
- Traducidos:
  17 tokens
- Frecuencias:
  - entero: 7
  - retornar: 3
  - si: 1
  - sino: 1
  - mientras: 1
  - para: 1
  - flotante: 1
  - caracter: 1
  - vacio: 1
- Strings/Comentarios:
  100% preservados

Conclusiones Técnicas y Repositorio

1. Eficiencia Computacional: El lenguaje C supera a Python por un factor de 15.3x en validaciones combinatorias de ajedrez y ofrece control total sobre la asignación en Heap.

2. Rigor en Gestión de Memoria: Implementación estricta de asignación dinámica (malloc) y liberación oportuna (free) en cada ejercicio, eliminando fugas de memoria.

3. Solución a Desbordamientos Numéricos: Superación exitosa de la barrera de 64 bits en Pascal mediante BigInt exacto para n=100 (29 dígitos decimales).

4. Ingeniería del Compilador: Construcción de un analizador léxico robusto capaz de diferenciar palabras reservadas de identificadores, cadenas de texto y comentarios.

5. Disponibilidad y Versionado: Todo el código fuente, benchmarks y scripts de compilación se encuentran alojados en GitHub:

https://github.com/Bryd3n/lyc-parsecore